Context:
Target is a globally renowned brand and a prominent retailer in the United States. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.
This particular business case focuses on the operations of Target in Brazil and provides insightful information about 100,000 orders placed between 2016 and 2018. The dataset offers a comprehensive view of various dimensions including the order status, price, payment and freight performance, customer location, product attributes, and customer reviews.
By analyzing this extensive dataset, it becomes possible to gain valuable insights into Target's operations in Brazil. The information can shed light on various aspects of the business, such as order processing, pricing strategies, payment and shipping efficiency, customer demographics, product characteristics, and customer satisfaction levels.

Dataset: https://drive.google.com/drive/folders/1TGEc66YKbD443nslRi1bWgVd238gJCnb
The data is available in 8 csv files:

1. customers.csv
2. sellers.csv
3. order_items.csv
4. geolocation.csv
5. payments.csv
6. reviews.csv
7. orders.csv
8. products.csv

Problem Statement:

Assuming you are a data analyst/ scientist at Target, you have been assigned the task of analysing the given dataset to extract valuable insights and provide actionable recommendations.

1. **Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:**

   A.Data type of all columns in the "customers" table.

Solution:

```
SELECT
  column_name,
  data_type
FROM
  `target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'customers';
```

Screenshot of Output:

Query results

Save results ▾ Open in ▾

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

Results per page: 50 ▾ 1 - 5 of 5

Insights :

1. The Output contains two data types of Customer Table.
2. One is String data Type and the other is Int64 Data Type.
3. Out of which 4 are of String Data Type And 1 is of Int64 Data Type.

B. Get the time range between which the orders were placed.

Solution :

```
SELECT
 MIN(order_purchase_timestamp) AS first_order_time,
 MAX(order_purchase_timestamp) AS last_order_time
 FROM
`target.orders`;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	first_order_time	last_order_time
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Insights :

1. The Time Range between which the orders were placed is **2016-09-04 21:15:19 UTC**(first order time) And **2018-10-17 17:30:18 UTC**(last order time)

C. Count the Cities & States of customers who ordered during the given period.

Solution :

```
SELECT
  COUNT(DISTINCT customer_city) AS total_cities,
  COUNT(DISTINCT customer_state) AS total_states
FROM
  target.customers AS c
JOIN
  target.orders AS o
ON
  c.customer_id = o.customer_id
WHERE
  o.order_purchase_timestamp BETWEEN (SELECT
  MIN(order_purchase_timestamp)
  FROM
  target.orders) AND (SELECT
  MAX(order_purchase_timestamp)
  FROM
  `target.orders`);
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	total_cities	total_states
1	4119	27

Insights :

1. The output shows a total of 4119 cities who ordered during the period.
2. The output also shows that there are total of 27 states who ordered during the period.

2.In-depth Exploration:

A. Is there a growing trend in the no. of orders placed over the past years?

Solution :

```
SELECT
  EXTRACT(YEAR FROM order_purchase_timestamp) AS Order_Year,
  COUNT(*) AS No_of_orders
FROM target.orders
GROUP BY Order_Year
ORDER BY Order_Year;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON

Row	Order_Year	No_of_orders
1	2016	329
2	2017	45101
3	2018	54011

Insights :

1. From the output we can see there is a growing trend in the total orders placed from the previous year.
2. From year 2016 to 2017 there is a steep jump in the no of orders from 329 orders in 2016 to 45101 orders in 2017.
3. There is also a rise in orders from 2017 to 2018 from 45101 orders in 2017 to 54011 orders in 2018.

B .Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Solution :

```
SELECT
  EXTRACT(MONTH FROM order_purchase_timestamp) AS Order_Month,
  EXTRACT(YEAR FROM order_purchase_timestamp) AS Order_Year,
  COUNT(*) AS No_of_orders
FROM target.orders
GROUP BY Order_Year,Order_Month
ORDER BY Order_Year,Order_Month;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON

Row	Order_Month	Order_Year	No_of_orders
1	9	2016	4
2	10	2016	324
3	12	2016	1
4	1	2017	800
5	2	2017	1780
6	3	2017	2682
7	4	2017	2404
8	5	2017	3700
9	6	2017	3245

Insights :

1. From the output we can clearly see there is seasonality in the no. of orders placed in different months of the year.
2. The highest orders are placed in October of 2016,November of 2017 and January of 2018.

C.During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

Solution :

```
SELECT
(CASE
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0
AND 6 THEN "Dawn"
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7
AND 12 THEN "Mornings"
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13
AND 18 THEN "Afternoon"
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19
AND 23 THEN "Night"
END) AS time_of_the_day,
COUNT(*) AS total_no_of_orders
FROM target.customers AS c
JOIN target.orders AS o
ON c.customer_id=o.customer_id
GROUP BY time_of_the_day
ORDER BY total_no_of_orders DESC;
```

Screenshot of Output:

Query results

Job information | Results | Visualization

Row	time_of_the_day	total_no_of_orders
1	Afternoon	38135
2	Night	28331
3	Mornings	27733
4	Dawn	5242

Insights :

1. From the output we can clearly see that the Brazilian customers mostly plays their orders in the Afternoon with total of 38135 orders.
2. And the least no of orders are placed at the Dawn with a total of 5242 orders.

3.Evolution of E-commerce orders in the Brazil region:

A. Get the month on month no. of orders placed in each state.

Solution :

```
SELECT
   EXTRACT(YEAR FROM order_purchase_timestamp) AS Order_year,

   EXTRACT(MONTH FROM order_purchase_timestamp) AS
Order_Month,
   COUNT(*) AS Total_No_of_orders,C.customer_state
FROM target.orders AS O
JOIN `target.customers`AS C
ON O.customer_id=C.customer_id
GROUP BY Order_year,Order_Month,C.customer_state
ORDER BY Order_year,Order_Month,Total_No_of_orders DESC;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON | Execution details

Row	Order_year	Order_Month	Total_No_of_orders	customer_state
1	2016	9	2	SP
2	2016	9	1	RS
3	2016	9	1	RR
4	2016	10	113	SP
5	2016	10	56	RJ
6	2016	10	40	MG
7	2016	10	24	RS
8	2016	10	19	PR
9	2016	10	11	SC

Insights :

- From the output we can see the month on month orders placed in each state in the year 2016,2017 and 2018.
- Some states have higher orders while others have low orders which are shown in the respective year and month ordered according to the no. of orders in descending order.

B. How are the customers distributed across all the states?

Solution :

```
SELECT customer_state,COUNT(DISTINCT customer_id) AS
Total_no_of_customers,
ROUND(100.0 * COUNT(DISTINCT customer_id) /
    (SELECT COUNT(DISTINCT customer_id) FROM
target.customers),
    2
  ) AS Perc_out_of_total
FROM `target.customers`
GROUP BY customer_state
ORDER BY Total_no_of_customers DESC;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON | Execution details | Exe

Row	customer_state	Total_no_of_cust...	Perc_out_of_total
1	SP	41746	41.98
2	RJ	12852	12.92
3	MG	11635	11.7
4	RS	5466	5.5
5	PR	5045	5.07
6	SC	3637	3.66
7	BA	3380	3.4
8	DF	2140	2.15
9	ES	2033	2.04

Insights :

1. From the output we can see that the highest percentage of customers are from state SP with a total of 41.98 %.
2. And the lowest percentage of customers are from state RR With a total of 0.05 %.

4.Impact on Economy: Analyse the money movement by e-commerce by looking at order prices, freight and others.

A. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

Solution :

```
 SELECT Year,Cost_of_order, ((LEAD(Cost_of_order,1) OVER(ORDER
BY YEAR)-Cost_of_order)/(Cost_of_order)*100) AS
Percent_inc_in_cost
FROM(

SELECT EXTRACT(YEAR FROM order_purchase_timestamp) AS Year,
   ROUND(SUM(P.payment_value),2) AS cost_of_order
   FROM target.orders AS O
   JOIN target.payments AS P
   ON O.order_id=P.order_id
   WHERE EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN
1 AND 8
   AND EXTRACT(YEAR FROM order_purchase_timestamp) BETWEEN
2017 AND 2018
GROUP BY EXTRACT(YEAR FROM order_purchase_timestamp));
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON

Row	Year	Cost_of_order	Percent_inc_in_co...
1	2017	3669022.12	136.9768716466...
2	2018	8694733.84	null

Insights :

- From the output we can see that there is a rise of 136.98 % in Orders from year 2017 to 2018(including months between orders from Jan to Aug only)

B. Calculate the Total & Average value of order price for each state.

Solution :

```
SELECT C.customer_state,
   ROUND(SUM(P.payment_value),2) AS
Total_Value,ROUND(AVG(P.payment_value),2) AS Avg_Value
   FROM target.orders AS O
   JOIN target.payments AS P
   ON O.order_id=P.order_id
   JOIN target.customers C
   ON O.customer_id=C.customer_id
GROUP BY C.customer_state
ORDER BY Total_Value DESC;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON | Execution details

Row	customer_state	Total_Value	Avg_Value
1	SP	5998226.96	137.5
2	RJ	2144379.69	158.53
3	MG	1872257.26	154.71
4	RS	890898.54	157.18
5	PR	811156.38	154.15
6	SC	623086.43	165.98
7	BA	616645.82	170.82
8	DF	355141.08	161.13
9	GO	350092.31	165.76

Insights :

- From the output we can see that SP has the highest Total value and RR has the lowest Total value.
- We can also see that PB has the highest Avg value and SP has the lowest Avg value.

C. Calculate the Total & Average value of order freight for each state.

Solution :

```
SELECT C.customer_state,
   ROUND(SUM(OI.freight_value),2) AS
Total_Value,ROUND(AVG(OI.freight_value),2) AS
Avg_Value
   FROM target.orders AS O
   JOIN target.order_items AS OI
   ON O.order_id=OI.order_id
   JOIN target.customers C
   ON O.customer_id=C.customer_id
GROUP BY C.customer_state
ORDER BY Total_Value DESC;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JSON | Execution details

Row	customer_state	Total_Value	Avg_Value
1	SP	718723.07	15.15
2	RJ	305589.31	20.96
3	MG	270853.46	20.63
4	RS	135522.74	21.74
5	PR	117851.68	20.53
6	BA	100156.68	26.36
7	SC	89660.26	21.47
8	PE	59449.66	32.92
9	GO	53114.98	22.77

Insights :

- From the output we can see that SP has the highest order freight and RR has the lowest order freight.
- We can also see that RR has the Highest Avg order freight and SP has the lowest Avg order freight.

5.Analysis based on sales, freight and delivery time.

A. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.
   Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
   Do this in a single query.

   You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:
   - **time_to_deliver** = order_delivered_customer_date - order_purchase_timestamp
   - **diff_estimated_delivery** = order_delivered_customer_date - order_estimated_delivery_date

Solution :

```
SELECT
DATE_DIFF(order_delivered_customer_date,order_purchase_timestamp,DAY) AS time_to_del,
DATE_DIFF(order_delivered_customer_date,order_estimated_delivery_date,DAY) AS diff_estimated_delivery

FROM target.orders
```

Screenshot of Output:

Query results

Save results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	time_to_del	diff_estimated_de...
1	35	-16
2	30	-28
3	30	12
4	54	36
5	43	-6
6	36	-14
7	29	-20
8	30	-19
9	44	-5

Results per page: 50 1 – 50 of 99441

Insights :

- From the output we can see the time to deliver the order and the different estimated delivery time .
- With negative diff_estimated_delivery showing orders delivered before the estimated delivery time and positive diff_estimated_delivery shows orders were delivered after the estimated delivery time.

Recommendation:

- We can reduce the diff_estimated_delivery time so that most customers order for their products to get the orders ddelivered on time.

B.Find out the top 5 states with the highest & lowest average freight value.

Solution :

```
A.)Top 5 States with highest freight value

   SELECT C.customer_state,
   ROUND(AVG(OI.freight_value),2) AS Avg_Freight_Value
   FROM target.orders AS O
   JOIN target.order_items AS OI
   ON O.order_id=OI.order_id
   JOIN target.customers C
   ON O.customer_id=C.customer_id
GROUP BY C.customer_state
ORDER BY Avg_Freight_Value DESC;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | J

Row	customer_state	Avg_Freight_Value
1	RR	42.98
2	PB	42.72
3	RO	41.07
4	AC	40.07
5	PI	39.15

A.)Top 5 States with lowest freight value

```
SELECT C.customer_state,
   ROUND(AVG(OI.freight_value),2) AS Avg_Freight_Value
   FROM target.orders AS O
   JOIN target.order_items AS OI
   ON O.order_id=OI.order_id
   JOIN target.customers C
   ON O.customer_id=C.customer_id
GROUP BY C.customer_state
ORDER BY Avg_Freight_Value ASC;
```

Screenshot of Output:

Query results

Job information | Results | Visualization

Row	customer_state	Avg_Freight_Value
1	SP	15.15
2	PR	20.53
3	MG	20.63
4	RJ	20.96
5	DF	21.04

Insights :

- From the output we can see that the Top 5 states with the highest freight value are RR.PB,RO,AC,PI.
- From the output we can see that the Top 5 states with the lowest freight value are SP,PR,MG,RJ,DF.

C.Find out the top 5 states with the highest & lowest average delivery time.

Solution :

```
A.)Top 5 States with highest average delivery time.

SELECT C.customer_state,
ROUND(AVG(DATE_DIFF(order_delivered_customer_date,order_purcha
se_timestamp,DAY)),2) AS Avg_del_time
   FROM target.orders AS O
   JOIN target.order_items AS OI
   ON O.order_id=OI.order_id
   JOIN target.customers C
   ON O.customer_id=C.customer_id
GROUP BY C.customer_state
ORDER BY Avg_del_time DESC
LIMIT 5;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | J

Row	customer_state	Avg_del_time
1	RR	27.83
2	AP	27.75
3	AM	25.96
4	AL	23.99
5	PA	23.3

A.)Top 5 States with lowest average delivery time.

```
SELECT C.customer_state,
ROUND(AVG(DATE_DIFF(order_delivered_customer_date,order_purcha
se_timestamp,DAY)),2) AS Avg_del_time
   FROM target.orders AS O
   JOIN target.order_items AS OI
   ON O.order_id=OI.order_id
   JOIN target.customers C
   ON O.customer_id=C.customer_id
GROUP BY C.customer_state
ORDER BY Avg_del_time ASC
LIMIT 5;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JS

Row	customer_state	Avg_del_time
1	SP	8.26
2	PR	11.48
3	MG	11.52
4	DF	12.5
5	SC	14.52

Insights :

- From the output we can see that the Top 5 states with the highest average delivery time are RR,AP,AM,AL,PA.
- From the output we can see that the Top 5 states with the lowest average delivery time are SP,PR,MG,DF,SC.

D. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Solution :

```
SELECT C.customer_state,

AVG(DATE_DIFF(order_delivered_customer_date,order_estimated_de
livery_date,DAY)) AS Del_time
   FROM target.orders AS O
   JOIN target.order_items AS OI
   ON O.order_id=OI.order_id
   JOIN target.customers C
   ON O.customer_id=C.customer_id
GROUP BY C.customer_state
ORDER BY Del_time ASC
LIMIT 5;
```

Screenshot of Output:

Query results

Job information | Results | Visualization | JS

Row	customer_state	Del_time
1	AC	-20.01098901098...
2	RO	-19.08058608058...
3	AM	-18.97546012269...
4	AP	-17.44444444444...
5	RR	-17.43478260869...

Insights :

- From the output we can see that the Top 5 states with the fast delivery are AC,RO,AM,AP,RR.
- With more negative value denoting how fast the order is delivered before the actual estimated delivery date.

6.Analysis based on the payments:

A.Find the month on month no. of orders placed using different payment types.

Solution :

```
SELECT P.payment_type,
   EXTRACT(MONTH FROM order_purchase_timestamp) AS Month,
   COUNT(O.order_id) AS No_of_orders
   FROM target.orders O
   JOIN target.payments P
   ON O.order_id=P.order_id
   GROUP BY Month,P.payment_type
   ORDER BY Month,No_of_orders  DESC;
```

Screenshot of Output:

Query results

Save results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	payment_type	Month	No_of_orders
1	credit_card	1	6103
2	UPI	1	1715
3	voucher	1	477
4	debit_card	1	118
5	credit_card	2	6609
6	UPI	2	1723
7	voucher	2	424
8	debit_card	2	82
9	credit_card	3	7707

Results per page: 50 1 – 50 of 50

Insights :

- From the output we can see the month on month order placed using different payment types like credit card,debit card,UPI,voucher.
- The highest order is placed in the month of May with 8350 orders and the payment type is credit card.
- The lowest order is placed in the month of Sep with 1 order and the payment type is not_defined.

B. Find the no. of orders placed on the basis of the payment installments that have been paid.

Solution :

```
SELECT P.payment_installments ,
```

```sql
COUNT(DISTINCT O.order_id) AS No_of_orders
FROM target.orders O
JOIN target.payments P
ON O.order_id=P.order_id
GROUP BY P.payment_installments
ORDER BY P.payment_installments;
```

Screenshot of Output:

Query results

Job information | Results | Visualization

Row	payment_installm...	No_of_orders
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253

Insights :

- For the output we can clearly see that most customers pay in the first installments based on the no. of orders .
- And then the second and third most preferred is 2 and in 3 installments based on the no.of orders.

- Very few people prefer more no. of installments.